Question 1. Copy/Paste the results of when you run benchmarking code in `BenchmarkForAutoComplete`. In that class code look at the instance variables for `BenchmarkForAutocomplete` and uncomment the two other implementation names so that `String[] myCompletorNames` has all three Strings: `"BruteAutocomplete"`, `"BinarySearchAutocomplete"`, and `"HashListAutocomplete"` (if you want to benchmark only a subset of these, perhaps because one isn't working, just leave it commented out).

Run `BenchmarkForAutocomplete` three times, once for each of the files you'll see in the Benchmark program: `threeletterwords.txt`, `fourletterwords.txt`, and `alexa.txt`. You can change which file is being used inside of the `doMark` method. **Copy and paste all three results into your analysis**. An example and detailed information about the output is described in the section below.

```
init time: 0.01641      for BruteAutocomplete

init time: 0.006093     for BinarySearchAutocomplete

init time: 0.05453      for HashListAutocomplete
```

search	size	#match	BruteAutoc	BinarySear	HashListAu
	17576	50	0.00308560	0.01312630	0.00001020
	17576	50	0.00047940	0.00265130	0.00001870
a	676	50	0.00036490	0.00014590	0.00000960
a	676	50	0.00025850	0.00016160	0.00001070
b	676	50	0.00019180	0.00012190	0.00001020
c	676	50	0.00026890	0.00039280	0.00002890
g	676	50	0.00016660	0.00010060	0.00000800
ga	26	50	0.00016350	0.00006100	0.00000910
go	26	50	0.00017140	0.00007360	0.00000970
gu	26	50	0.00016640	0.00007910	0.00001010
x	676	50	0.00017480	0.00011170	0.00000970
y	676	50	0.00029290	0.00012140	0.00001180
z	676	50	0.00035550	0.00021350	0.00001720
aa	26	50	0.00037880	0.00018930	0.00001850
az	26	50	0.00029990	0.00011420	0.00001780
za	26	50	0.00028960	0.00007950	0.00001550

```
zz      26      50      0.00025090      0.00005960      0.00005090

zqzqwwx 0       50      0.00035500      0.00005520      0.00001180

size in bytes=246064     for BruteAutocomplete

size in bytes=246064     for BinarySearchAutocomplete

size in bytes=1092468    for HashListAutocomplete
```

```
init time: 0.04697      for BruteAutocomplete

init time: 0.02511      for BinarySearchAutocomplete

init time: 0.8789       for HashListAutocomplete

search  size    #match  BruteAutoc      BinarySear      HashListAu

        456976  50      0.01427900      0.05865800      0.00001350

        456976  50      0.00784860      0.02671880      0.00002240

a       17576   50      0.01263350      0.00115030      0.00001680

a       17576   50      0.00949280      0.00133000      0.00001700

b       17576   50      0.00838020      0.00121850      0.00001670

c       17576   50      0.00909040      0.00111080      0.00001620

g       17576   50      0.00797060      0.00122160      0.00001780

ga      676     50      0.00758930      0.00019000      0.00001660

go      676     50      0.00726730      0.00012270      0.00001560

gu      676     50      0.00755380      0.00012430      0.00001560

x       17576   50      0.00695850      0.00103440      0.00001590

y       17576   50      0.00717240      0.00106410      0.00001510

z       17576   50      0.00817300      0.00108940      0.00002270

aa      676     50      0.00777330      0.00014160      0.00001710

az      676     50      0.00738030      0.00010590      0.00001620

za      676     50      0.00727410      0.00010430      0.00001720

zz      676     50      0.00768210      0.00010380      0.00001690

zqzqwwx 0       50      0.00628830      0.00007860      0.00001110

size in bytes=7311616    for BruteAutocomplete
```

```
size in bytes=7311616    for BinarySearchAutocomplete

size in bytes=40322100   for HashListAutocomplete
```

```
init time: 0.04697      for BruteAutocomplete

init time: 0.02511      for BinarySearchAutocomplete

init time: 0.8789       for HashListAutocomplete
```

search	size	#match	BruteAutoc	BinarySear	HashListAu
	456976	50	0.01427900	0.05865800	0.00001350
	456976	50	0.00784860	0.02671880	0.00002240
a	17576	50	0.01263350	0.00115030	0.00001680
a	17576	50	0.00949280	0.00133000	0.00001700
b	17576	50	0.00838020	0.00121850	0.00001670
c	17576	50	0.00909040	0.00111080	0.00001620
g	17576	50	0.00797060	0.00122160	0.00001780
ga	676	50	0.00758930	0.00019000	0.00001660
go	676	50	0.00726730	0.00012270	0.00001560
gu	676	50	0.00755380	0.00012430	0.00001560
x	17576	50	0.00695850	0.00103440	0.00001590
y	17576	50	0.00717240	0.00106410	0.00001510
z	17576	50	0.00817300	0.00108940	0.00002270
aa	676	50	0.00777330	0.00014160	0.00001710
az	676	50	0.00738030	0.00010590	0.00001620
za	676	50	0.00727410	0.00010430	0.00001720
zz	676	50	0.00768210	0.00010380	0.00001690
zqzqwwx	0	50	0.00628830	0.00007860	0.00001110

```
size in bytes=7311616    for BruteAutocomplete

size in bytes=7311616    for BinarySearchAutocomplete

size in bytes=40322100   for HashListAutocomplete
```

Question 2. Let `N` be the total number of terms, let `M` be the number of terms that prefix-match a given `search` term (the `size` column above), and let `k` be the number of highest weight terms returned by `topMatches` (the `#match` column above). The runtime complexity of `BruteAutocomplete` is `O(N + M log(k))`. The runtime complexity of `BinarySearchAutocomplete` is `O(log(N) + M log(k))`. Yet you should notice (as seen in the example timing above) that `BruteAutocomplete` is similarly efficient or even slightly more efficient than `BinarySearchAutocomplete` on the empty `search` String `""`. Answer the following:

- Explain why this observation (that `BruteAutocomplete` is similarly efficient or even slightly more efficient than `BinarySearchAutocomplete` on the empty `search` String `""`) makes sense given the values of `N` and `M` -- in other words, is this an anomaly based on small clocktimes or is there a reason for it given the values of N, M, and what will happen with BinarySearch when searching for the empty string? (Hint: how many terms match the empty prefix?)

  say empty string, there is 1 prefix (an empty string), and 1 match, also an empty string. O(N + M log(k)) = O(1 + 1logk), O(logN + M log K) = O(1 + 1logk). Both are same asymptotically, however, some overhead factors in binary can make it slower.

Question 3. Run the `BenchmarkForAutocomplete` again using `alexa.txt` but doubling `matchSize` to `100` (`matchSize` is specified in the `runAM` method). Again copy and paste your results. Recall that `matchSize` determines `k`, the number of highest weight terms returned by `topMatches` (the `#match` column above). Do your data support the hypothesis that the dependence of the runtime on `k` is logarithmic for `BruteAutocomplete` and `BinarySearchAutocomplete`?

```
init time: 0.2314       for BruteAutocomplete

init time: 1.392        for BinarySearchAutocomplete

init time: 5.204        for HashListAutocomplete

search  size    #match  BruteAutoc      BinarySear      HashListAu

        1000000 100     0.01625510      0.07611100      0.00001560

        1000000 100     0.00970140      0.05279050      0.00002330

a       69464   100     0.00977760      0.00417640      0.00001590

a       69464   100     0.00898310      0.00373800      0.00001430

b       56037   100     0.00945970      0.00426120      0.00003300

c       65842   100     0.00961880      0.00401740      0.00001630

g       37792   100     0.00935180      0.00243650      0.00001610

ga      6664    100     0.00960350      0.00055370      0.00001720

go      6953    100     0.00931420      0.00057660      0.00001850

gu      2782    100     0.01111020      0.00035510      0.00002870

x       6717    100     0.00971070      0.00071570      0.00002620

y       16765   100     0.00842460      0.00112740      0.00001790

z       8780    100     0.00853570      0.00065390      0.00002030

aa      718     100     0.00952730      0.00013760      0.00001770

az      889     100     0.00959150      0.00014980      0.00001780

za      1718    100     0.00925920      0.00036250      0.00002910

zz      162     100     0.00865940      0.00009130      0.00001840

zqzqwwx 0       100     0.00916330      0.00007670      0.00001180

size in bytes=38204230   for BruteAutocomplete

size in bytes=38204230   for BinarySearchAutocomplete
```

```
size in bytes=475893648  for HashListAutocomplete
```

The results are consistent with logarithmic dependence on k, although they do not conclusively establish it. For example, increasing k from 50 to 100 increased BinarySearchAutocomplete's time for "gu" from about 0.0001655 to 0.0003551 seconds, while BruteAutocomplete stayed near 0.011 seconds. Other measurements varied, with some decreasing.

Doubling k increases $\log_2(k)$ from about 5.64 to 6.64, so we expect a modest increase in priority-queue work rather than consistently doubled total runtime. BruteAutocomplete must also scan all one million terms regardless of k, which can hide that increase. Timing noise makes the overall evidence mixed.

Question 4. Briefly explain why `HashListAutocomplete` is much more efficient in terms of the empirical runtime of `topMatches`, but uses more memory than the other `Autocomplete` implementations.

If we look at topMatches, its implementation is essentially O(1) runtime since everything is cached. But the sacrifice for this is that setting up the caching requires more memory, which is why HashListAutoComplete is asymptotically faster, but requires more memory.